

StagNest — Website Build Spec (static site, no backend)

Single source of truth for building the StagNest **website** — plain HTML + CSS + vanilla JavaScript, **no build step, no npm, no server, no database**. Open `index.html` and it works. Paste this into **Claude Code** to generate it, feed §4 **Design System** + §5 **Pages** into **Claude's design tools**, and keep it open in **VS Code** to make it functional.

This is a front-end website: data is sample content in a JS file and all interactions are client-side (favorites saved in the browser). It is NOT a real app with accounts, a database, or real payments — if you need those, that's a separate Next.js project. See §12.

1. Product

StagNest — a verified off-campus housing marketplace for **Fairfield University** students. Marketing + browse experience: students discover houses, filter by what matters (walk to campus/bars, rent, beds), see location ratings, and (in the demo) sign in, save favorites, and explore dashboards. Brand promise: verified students only, real reviews, no scams.

Roles represented (as demo screens, not real auth): student, landlord, admin.

2. How to use this spec

- **Claude Code:** "Build the static website in this spec. Start with §9 Build Plan, Phase 0 (shared assets), then the pages in §5. Use the exact tokens in §4. No frameworks, no build step — plain `.html` + one `.css` + vanilla JS in `assets/`."
 - **Claude design tools:** feed §4 (tokens, components, logo) and §5 (per-page layout) to design screens that match the tokens exactly.
 - **VS Code:** open the folder, edit files, and preview with the **Live Server** extension (right-click `index.html` → "Open with Live Server"). No terminal or install needed. §10 is your definition of done.
-

3. Tech (deliberately minimal)

- **HTML5** — one file per page.
 - **CSS** — a single hand-written `assets/styles.css` (design system + components). No Tailwind, no build.
 - **Vanilla JavaScript** — `assets/data.js` (sample data + config) and `assets/app.js` (shared behavior). No modules, no bundler — plain `<script>` tags and global functions so it runs when opened from disk.
 - **External resources (optional, degrade gracefully)**: Google Fonts (Plus Jakarta Sans) and, only if a key is provided, the Google Maps JS API. Everything works offline without them.
 - **Persistence**: `localStorage` for favorites. No server, no database.
-

4. Design system

Brand: clean, trustworthy, collegiate (Airbnb warmth + Zillow density + Notion calm). Logo is a **stag** mark (antlered stag head, inline SVG, `currentColor`) in a rounded-square red badge; wordmark "StagNest" with a small uppercase "Fairfield housing" kicker.

Colors — CSS variables in `:root`:

--red #BA0C2F (Fairfield cardinal / PMS 200 — primary actions, brand)

--red-dark #8E0923 (hover)

--red-bg #FCEAEE (icon tiles, tints)

--ink #16130F (primary text)

--muted #625C54 (secondary text)

--faint #9A948C (hints/captions)

--bg #FBFAF8 (page background)

--surface #FFFFFF (cards)

--surface-2 #F3F1EC (inset surfaces)

--border #E7E3DB (hairlines)

--ink-2 #201C17 (dark footer)

Status: --emerald #0F6E56 · --blue #1F5FA6 · --amber #8A5206 (+ their light bg tints)

--radius 14px · --shadow 0 10px 30px rgba(22,19,15,.07)

--font "Plus Jakarta Sans", -apple-system, system-ui, sans-serif

Typography: Plus Jakarta Sans, weights 400/500/600/700/800. Sentence case everywhere. Hero H1 ~58px/800, section H2 ~30px/800, H3 ~18px/700, body 15–16px/400.

Shape/spacing: card radius 14px; controls 10–12px; section padding ~84px vertical; card padding 20px; gaps 12–24px. Soft shadow on cards only.

Icons: line SVG (24px, 1.7 stroke, `currentColor`) — **never emoji as UI icons.**

Reusable CSS component classes (define once in `styles.css`): `.btn` (+ `.btn-primary/.btn-dark/.btn-ghost`, `.btn-sm/.btn-lg/.btn-block`), `.card/.card-pad`, `.badge` (green/blue/amber/red/gray), `.chip`, `.input/select/textarea`, `.tabs/.tab`, `.listing` (card), `.bar` (rating), `.switch` (toggle), `.nav`, `footer.site` (dark, multi-column), `.hero`, `.eyebrow`, `.feature` `.ic` (icon tile).

5. Pages & file structure

index.html	Landing (marketing)
browse.html	Search + filters + list/map
listing.html	Property detail (reads ?id=)
about.html	About + FAQ
list-property.html	Landlord listing form + confirmation
signin.html	Fairfield-email sign-in (demo)
dashboard.html	Student: saved / applications / documents / alerts
rental.html	Tenant portal: rent / maintenance / lease / messages

manage.html Landlord: rent roll / maintenance / applications / listings

admin.html Admin: overview / landlords / listings / reviews / users (NOT linked in nav)

assets/styles.css Design system + all components

assets/data.js window.CONFIG (Maps key, campus) + window.LISTINGS + window.money

assets/app.js Stag logo SVG, favicon, nav+footer injection, favorites, listing-card renderer

Every page: links `assets/styles.css` in `<head>`; has `<header id="nav"></header>` at top and `<footer id="footer"></footer>` at bottom (app.js fills them); loads `assets/data.js` then `assets/app.js` before its own inline `<script>`.

Per-page content:

- **index** — hero (headline "Every Fairfield house, one trusted place.", email CTA, trust line), featured listings (3 cards from data.js), 6 feature cards (SVG icons: verified students, location ratings, talk to renters, universal application, pay rent + split, trusted reviews), how-it-works (3 steps), student/landlord split, closing CTA.
- **browse** — sticky filter rail (rent min/max, beds, max-walk-to-campus slider, furnished, pets); List ⇌ Map toggle; results grid of listing cards. Map = Google Maps if `CONFIG.MAPS_KEY` set, else a schematic price-pin map.
- **listing** — reads `?id=`; photo grid, key facts (beds/baths/available/lease), description, Location Ratings panel (walk to campus/bars/restaurants/grocery as minutes + bars; safety/quiet/ convenience), talk-to-renter, amenities, sticky sidebar (landlord card, deposit, Apply / Save / Message).
- **signin** — student/landlord toggle; step 1 email (students must end in `@student.fairfield.edu`, validated client-side); step 2 six-digit code (demo — any 6 digits) → routes to dashboard or manage.
- **dashboard** — tabs: Saved (favorited listings), Applications (status badges), Documents (checklist), Alerts (saved searches with toggles).
- **rental** — tabs: Rent (balance, Pay button clears it, autopay toggle, roommate split, history), Maintenance (new request + list), Lease & docs, Messages thread.
- **manage** — stat header; tabs: Rent roll (collection % + tenant table), Maintenance queue (status buttons), Applications (accept/decline), Listings (views/saves). "Add a listing" → `list-property.html`.
- **list-property** — form (basics, details, utilities, parking, photos w/ thumbnail preview, contact) → submit swaps to a confirmation screen with a summary.
- **about** — mission, values (SVG icons), expandable FAQ, CTA.
- **admin** — tabs: Overview (stat cards), Landlords (approve/reject), Listings (remove flagged), Reviews (publish/remove), Users. Reachable only by opening `admin.html` directly (see §11).

6. Shared JS contract (`data.js` + `app.js`)

`data.js` exposes globals:

- `window.CONFIG = { MAPS_KEY: "", CAMPUS: { lat, lng } }` — paste a Google Maps key to enable the live map.
- `window.LISTINGS = [ { id, title, address, lat, lng, rent, beds, baths, furnished, pets, avail, lease, deposit, parking, color, verified, landlord, resp, desc, amenities:[], r:{campus,bars,rest,grocery, safety,quiet,conv} } ]` — the sample dataset.
- `window.money = (n) => "$" + n.toLocaleString()`.

`app.js` exposes/handles:

- `STAG` — inline stag-head SVG string (uses `currentColor`).
- `renderChrome()` on `DOMContentLoaded` — injects the sticky nav (brand + links + Sign in) and the dark multi-column footer into `#nav/#footer`, marks the active link, fills any `.stag-slot`, sets the favicon.
- Favorites: `getFavs()`, `isFav(id)`, `toggleFav(id)` backed by `localStorage["stagnest_favorites"]`.
- `listingCard(listing)` → HTML string for a listing card (photo, save heart, verified badge, price, chips, meta line). `onSave(btn, id)` toggles the heart and calls `window.onFavChanged` if a page defines it.
- The nav list intentionally **excludes** `admin.html`.

7. Behavior / interactions (what "functional" means here)

- Browse filters update the grid live; List/Map toggle switches views.
- The ♥ save button persists across pages via `localStorage` and appears on the Dashboard → Saved tab (page sets `window.onFavChanged = render` to re-render).
- Sign-in enforces the `@student.fairfield.edu` gate client-side, then routes by role.
- Tenant "Pay rent" clears the balance; maintenance/admin/landlord action buttons update state in place.
- List-property submit validates required fields, then shows a confirmation summary.
- Map uses Google Maps when `CONFIG.MAPS_KEY` is set (price markers + info windows) and falls back to the schematic map otherwise, including if the key errors.

8. Config

- **Google Maps (optional):** set `CONFIG.MAPS_KEY` in `assets/data.js` to a Google Maps JS API key. Blank = schematic map (still fully usable, works offline).
 - **Campus point:** `CONFIG.CAMPUS` = Fairfield University { `lat: 41.1553`, `lng: -73.2586` }.
-

9. Build plan (order for the agent)

- **Phase 0 — Shared foundation:** `assets/styles.css` (tokens §4 + all component classes), `assets/data.js` (CONFIG + LISTINGS + money), `assets/app.js` (STAG logo, favicon, nav + dark footer injection, favorites, `listingCard`). Nothing renders correctly until these exist.
 - **Phase 1 — Public pages:** index, browse (filters + map fallback), listing (`?id=`), signin, about, list-property.
 - **Phase 2 — Account pages:** dashboard, rental, manage, admin — each with client-side tabs and actions.
 - **Phase 3 — Polish:** responsive (nav collapses to a menu button; grids stack at 860px/560px), favicon, keep admin out of the nav, README.
-

10. Definition of done

- ☐ Opening `index.html` directly in a browser renders the full site (nav + footer injected, fonts, logo).
- ☐ Every page links `styles.css` and loads `data.js` then `app.js`.
- ☐ Browse filters (rent, beds, walk-to-campus, furnished, pets) change results; List/Map toggle works.
- ☐ Saving a listing persists after refresh and shows on Dashboard → Saved.
- ☐ `listing.html?id=2` shows that listing's detail.
- ☐ Sign-in rejects a non-`@student.fairfield.edu` email; a valid one routes into the site.
- ☐ Tenant "Pay rent", landlord maintenance actions, and admin moderation buttons visibly update.
- ☐ Stag logo + Fairfield cardinal red + Plus Jakarta Sans throughout; no emoji used as UI icons.
- ☐ Responsive down to phone width; nav collapses to a menu.

11. Deploy + the admin caveat

Deploy: it's just files. Drag the folder onto **Netlify Drop** (app.netlify.com/drop), or push to GitHub and enable **GitHub Pages**, or upload to any static host. That's the whole deploy.

Admin is not secure on a static site. Hiding `admin.html` from the nav only stops casual discovery — anyone who opens the file or views source sees it. There is no way to truly protect it client-side. If you publish the site and want admin gone, **delete `admin.html` before uploading**. Real, private admin requires a backend (the app version).

12. Static website vs the app (choose consciously)

This spec produces a **front-end website**: fast, free, zero-setup, great as a marketing site + interactive demo. It cannot do real logins, store data for other users, or take real rent — those are illusions here (sample data + `localStorage`). When you need real accounts, a database, verified sign-in, and Stripe rent payments, that's the **Next.js app** (a separate spec/codebase). Many teams ship the website first for marketing, then point its "Sign in" at the real app once it's live.